

Informatique 2 : Projet Sherlock

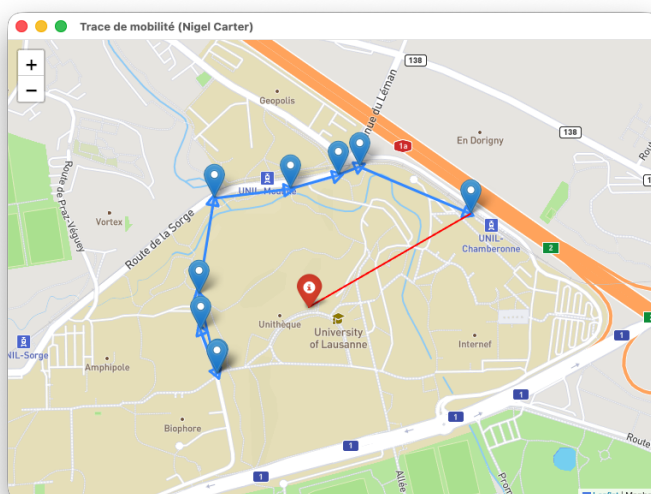
Instructions pour la réalisation du projet “Collecte et analyse de traces de mobilité dans le cadre d’une investigation”.

Lire attentivement les énoncés des exercices et tester chaque étape. Si besoin, penser à utiliser des copies (superficielles ou profondes) lors du retour ou du passage par références pour éviter les modifications intempestives.

Pour faciliter la correction du projet et des tests, respecter les noms et signatures (nombre et ordre des paramètres) des fonctions et des méthodes !

Le but général du projet est d’identifier, à partir de leurs traces de mobilité, les suspect-e-s plausibles pour un crime ayant eu lieu à un endroit donné, à une date et heure données. La méthodologie générale est la suivante¹ :

- Récupérer les différentes informations fournies au programme par l’enquêteur-trice. Ces données prennent la forme d’arguments de la ligne de commande, de fichiers de configuration et de fichiers de données.
- Lister les suspect-e-s et pour chacun d’entre eux, construire sa trace de mobilité (des informations sous forme de date et heure/localisation) à partir des sources de données spécifiées. Les sources sont des photographies géo-tagguées, des logs de connexion Wi-Fi contenant des données de localisation, et/ou des logs GPS de téléphones portables.
- Extraire de ces traces de mobilité les données de localisation de chaque suspect-e juste avant et juste après le crime (la dernière information de localisation antérieure au crime et la première postérieure au crime).
- Déterminer, à l’aide des données de cartographie d’un système d’information géographique, si le suspect a pu se rendre sur le lieu du crime depuis sa localisation juste avant le crime et du lieu du crime à sa localisation juste après le crime durant l’intervalle de temps correspondant.



Dans ce document, toutes les étapes seront décrites pas à pas pour parvenir à ce résultat. Le document est structuré en sections qui décrivent le travail à effectuer pour chaque module. Le numéro et le titre de la séance du cours où les notions nécessaires à l’implémentation du module correspondant sont indiqués en en-tête.

Attention

Les modules ne sont pas à traiter dans l’ordre ; le module [Section IV](#) peut d’ores et déjà être implémenté, mais pas le module [Section V^a](#). Dans un premier temps, commencer par intégrer les morceaux du projet qui ont déjà été effectués pendant les séances de TPs. Certaines sections

1. On notera que la méthodologie n’est en rien rigoureuse. Le but du projet est d’appliquer les concepts vus en cours à un exemple concret proche de la thématique de la formation.

contiennent des diagrammes UML donnant de **nombreuses informations** pour guider l'implémentation (**respecter les noms et types des attributs et des méthodes**). Ce document est accompagné d'un squelette du programme contenant des commentaires et des `TODO` indiquant le code à produire aux différents endroits du programme. Les mentions  **Urgent** indiquent les instructions minimums qui sont nécessaires pour avoir du code fonctionnel.

Chaque module contient un bloc `main` contenant une série d'instructions de tests. Ces tests doivent fonctionner une fois que le module est correctement complété. Il est indiqué, sous chaque bloc `main`, ce que le programme devrait afficher lorsque le bloc `main` est exécuté. Dé-commenter les instructions une fois que les méthodes correspondantes ont été implémentées.

Il convient également de tester vos méthodes pour gérer les erreurs posées par les cas problématiques, les cas particuliers, et les exceptions en général. Pour les exceptions, générer un message sur la sortie erreur si l'utilisateur·trice a spécifié l'argument *verbose* dans la ligne de commande.

a. Lire l'énoncé **en entier** afin d'identifier les différentes parties du programme et l'ordre dans lequel vous allez implémenter les différents modules.

Avant de commencer, télécharger [ce fichier ZIP](#) depuis Moodle, contenant les données, les dépendances et le squelette du projet, puis créer un nouveau projet PyCharm appelé *ProjetSherlock*. Placer les dossiers de données et le squelette dans votre projet. Dans PyCharm, renommer `src_skeleton` en `src`.

Attention

Votre arborescence devrait correspondre à ceci.

```
ProjetSherlock/
├── .venv/
├── requirements.txt
├── data/
│   ├── db/
│   ├── logs/
│   ├── pics/
│   ├── suspects.json
│   └── suspects.xml
└── src/
    ├── __init__.py
    ├── configuration.py
    ├── location.py
    ├── logs.py
    ├── picture.py
    ├── sherlock.py
    ├── suspects.py
    ├── utils.py
    └── wifi.py
```

Conseil

Si vous travaillez en équipe et votre projet se trouve dans un dossier synchronisé par OneDrive, **n'y créez pas d'environnement virtuel partagé** (par ex. `.venv/` dans le dossier du projet). Sinon, les fichiers de l'environnement peuvent être synchronisés et écrasés entre les deux membres du groupe. Dans ce cas, créez plutôt un environnement virtuel **hors** du dossier OneDrive, puis configurez PyCharm pour utiliser cet interpréteur (coin en bas à droite / Add Interpreter). Vous pouvez créer un environnement virtuel avec la commande suivante : `python3 -m venv sherlockenv`

L'interpréteur `python` de cet environnement virtuel se trouve dans l'arborescence suivante :

```
sherlockenv/
├── bin/
│   └── python
```

```
python3
python3.14
```

Vous pouvez installer les dépendances à l'aide du terminal dans PyCharm en utilisant la commande : `pip install -r requirements.txt` dans votre environnement virtuel. Il faut placer le fichier `requirements.txt` à la racine de votre projet pour que PyCharm puisse le détecter et vous demander d'installer automatiquement tous les modules requis. Il n'est pas permis d'installer d'autres dépendances que celles figurant dans le fichier `requirements.txt`.

Vous pouvez également installer les dépendances une par une dans PyCharm (PyCharm → en bas à gauche Python Packages ) : *certifi*, *exifread*, *folium*, *lxml*, *pyside6* (ces dépendances sont déjà installées sur les machines des salles de TP).

Section I Le module outil : `utils.py` Cours 1 : Manipulation avancée des objets de base

Le module `utils.py` contient des fonctions utiles pour le reste du programme.

Pour afficher les objets composites (voir [Section VI](#)), on a besoin d'une fonction `indent` pour décaler (indenter) le texte automatiquement en y ajoutant une chaîne de caractères en début de ligne.

Instructions :

1. Implémenter une fonction `indent` qui prend en paramètre un texte (sous forme d'une chaîne de caractères) et un espacement (sous forme d'une chaîne de caractères, égal à une tabulation par défaut) et qui ajoute le caractère d'espacement au début de chaque ligne du texte (TP1 exercice 14). Par exemple,

```
print(f"zero\n{indent('one\n' + indent('two\nthree', '\t-'))}") affiche :
zero
    one
        -two
        -three
```

Conseil

- Découper la chaîne de caractères passée en paramètre au niveau des passages à la ligne (`split`),
- Ajouter la chaîne d'espacement au début de chaque chaîne de caractères de la liste ainsi obtenue (`map` ou une liste en compréhension)
- Assembler les différentes chaînes de la liste en intercalant un passage à la ligne entre elles (`join` ou `reduce`).

Note : Dans l'exemple, les caractères `"\t"` et `"\n"` représentent respectivement une tabulation et un retour à la ligne.

Section II Le module principal : `sherlock.py` Cours 2 : Programmes avancés

Le module principal est contenu dans le fichier `sherlock.py` qui doit être exécuté pour mener à bien l'investigation complète. À la fin du projet, c'est ce fichier qu'il faudra exécuter pour lancer le programme d'investigation. L'enquêteur-trice peut, ou doit, passer un certain nombre d'informations au programme via des arguments de ligne de commande, à savoir :

- Un argument permettant de spécifier si les messages d'avertissement en cas de problèmes doivent être affichés. Il s'agit d'un argument **optionnel** booléen `-v/-verbose`, qui vaut `False` par défaut (voir cours et TP);
- Un argument **obligatoire** spécifiant le nom du fichier (*XML/JSON*) décrivant les suspect-e-s (voir [Section V](#));
- Un argument **obligatoire** spécifiant la clé pour l'API du système d'information géographique (SIG) (voir [Section X](#));
- Des arguments **obligatoires** spécifiant les coordonnées GPS (latitude et longitude, de type `float`) du lieu du crime;
- Un argument **obligatoire** spécifiant, sous forme textuelle, la date et heure du crime. Le format de la date devra être : `JJ/MM/AAAA-hh:mm:ss`, par exemple `17/03/2026-15:52:31`.

Instructions :

1. Implémenter, à l'aide du module `argparse`, la récupération des informations à partir de la ligne de commande. À titre d'indication, le message d'aide du module `sherlock` (obtenu en exécutant la commande `sherlock.py -h`) devrait être le suivant² :

```
usage: sherlock.py [-h] [-v] -s SUSPECTS -g GEO_API_KEY -lat LATITUDE
                  -lng LONGITUDE -d DATE

Identifie les suspect.e.s les plus plausibles à partir de leurs traces de
mobilité (issues de sources multiples incluant les localisations contenues
dans les traces Wi-Fi et les flux de photos géo-tagguées) pour un
crime spécifié par une date/heure et une localisation.

options:
  -h, --help                show this help message and exit
  -v, --verbose             affiche les détails de l'exécution du
                           programme et les avertissements.
  -s, --suspects SUSPECTS  fichier contenant la liste des suspect.e.s et les
                           sources de données de localisation.
  -g, --geo-api-key GEO_API_KEY
                           clé pour l'accès à l'API du SIG.
  -lat, --latitude LATITUDE
                           latitude de la scène du crime.
  -lng, --longitude LONGITUDE
                           longitude de la scène du crime.
  -d, --date DATE          date et heure du crime (au format JJ/MM/AAAA-hh:mm,
                           par exemple 17/03/2026-15:52:31).
```

Conseil

Penser à spécifier les types des arguments qui ne sont pas des chaînes de caractères de sorte qu'`argparse` fasse les vérifications et les conversions nécessaires. Réutiliser le code implémenté pendant les séances de TP (TP2).

2. Ecrire la ligne de commande complète permettant d'exécuter le module principal `sherlock` :
 - Clé API : Cette information vous sera transmise ultérieurement.
 - Crime :
 - Date du crime : **17/03/2026 à 15:52:31 (UTC+2)**
 - Lieu : **Unicentre, 1015 Ecublens, Suisse (46.522735, 6.580811)**
 - Déterminer les autres paramètres.
3. Créer un objet `Location` décrivant le lieu du crime à partir de ses coordonnées (obtenues depuis la ligne de commande) (voir [Section IV](#))
4. Convertir la date du crime (obtenue depuis la ligne de commande) en `datetime`.

Conseil

Utiliser la méthode `datetime.strptime(arg)` pour créer un objet `datetime` à partir d'une chaîne de caractères `arg`.

Les questions suivantes pourront seulement être traitées plus tard dans l'avancement du projet.

5. Ajouter le paramètre `verbose` (obtenu depuis la ligne de commande) à l'objet de configuration (voir [Section III](#)).
6. Définir la clé pour l'API de Google Maps (obtenue depuis la ligne de commande) dans son module (voir [Section X](#)).
7. Afficher un message de la forme suivante pour signaler le début de l'analyse. Le nom du lieu du crime aura été obtenu automatiquement à l'aide de la méthode adéquate qui repose sur l'API du SIG (voir [Section X](#)). Seule 5 décimales seront affichées pour les coordonnées.
Par exemple : "Investigation liée au crime du 17/03/2026 à 15:52:31 @ Unicentre, 1015 Ecublens, Suisse (46.52273,6.58081)"
8. Créer la liste des objets `Suspect` à l'aide de la méthode adéquate (voir [Section V](#)) à qui on fournira le nom du fichier de configuration `XML/JSON` (obtenu depuis la ligne de commande).

2. Pour faciliter la correction, utiliser les mêmes noms pour les différents arguments.

Conseil

La ligne de commande se présente sous la forme suivante (python3 sur MacOS) :

```
python sherlock.py -v -s ../data/suspects.xml ...
```

9. Pour chaque suspect-e, déterminer, à l'aide de la méthode adéquate qui repose sur l'API cartographique (voir [Section X](#)), si le suspect-e a pu se rendre sur le lieu du crime depuis sa localisation juste avant le crime et du lieu du crime à sa localisation juste après le crime durant l'intervalle de temps correspondant. **On considérera que le temps minimal réel (à pied) est la moitié du temps renvoyé par l'API cartographique.** Afficher la liste des suspect-e-s plausibles.
10. Intercepter toutes les exceptions possibles et afficher un message de la forme suivante (les points de suspension dénotent le message spécifique de l'erreur). En cas d'exception, interrompre l'exécution du programme et afficher dans tous les cas un message indiquant la fin de l'exécution : "[Erreur] L'erreur suivante est survenue durant l'exécution du programme : ...".

Attention

Évitez d'englober immédiatement tout le code du bloc main dans `sherlock.py` avec un `try/except`, car cela compliquera considérablement le débogage du projet. Il faut placer les `try/except` uniquement autour des lignes de code spécifiques qui pourraient générer une erreur.

Si vous souhaitez quand même mettre tout le code dans `sherlock.py` dans un `try/except`, importez le module `traceback` et appelez la fonction `traceback.print_exc()` dans le bloc `except` (uniquement si l'utilisateur-trice a spécifié l'argument `--verbose`). Ceci va afficher la trace d'exécution et le message d'erreur détaillé.

Section III La classe *Configuration* : `configuration.py` Cours 6 : UML et Patrons de conception

La classe `Configuration` permet de manipuler les paramètres de configuration globale du programme (p.ex. 'verbose'). On décrit ces paramètres sous la forme de couples clé-valeur (p. ex. "`verbose`": `True`). Cette classe doit être définie dans le module `configuration.py`.

Instructions :

1. Créer une classe `Configuration` contenant : une structure de données adéquate (initialement vide) permettant de stocker les couples clé-valeur, une méthode `add_element` permettant d'ajouter un élément de configuration, une méthode `get_element` permettant de récupérer la valeur d'un paramètre à partir de la clé correspondante.

>_ Exemple

Si on effectue `add_element('N', 4)` avec la structure de données contenant `{'verbose': False}`, on obtient : `{'verbose': False, 'N': 4}`

2. Ajouter un argument optionnel à `get_element` spécifiant la valeur de retour (judicieuse) si la clé spécifiée n'est pas présente dans la configuration.

>_ Exemple

En appelant `get_element('M', 7)` sur la structure de données précédente, on doit obtenir 7.

3. Implémenter la méthode `__str__` afin d'afficher la configuration et ses attributs.
4. Implémenter le patron `Singleton` pour cette classe pour permettre de manipuler la configuration globale de manière cohérente et facilement partout dans le programme.
5. Tester.

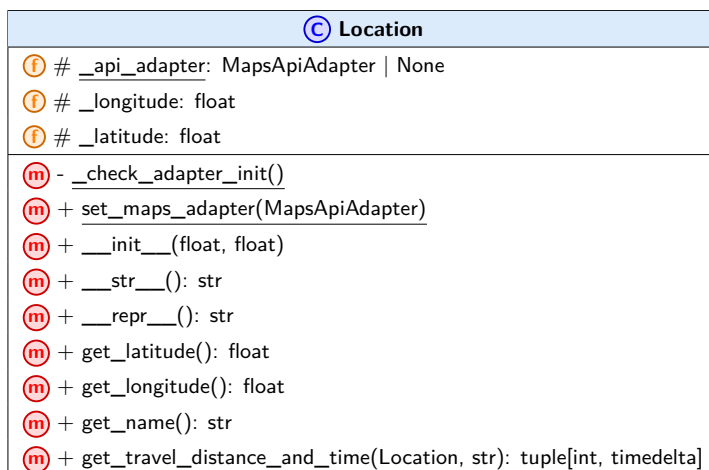
Section IV 📍 Les classes `Location`, `LocationSample` et `LocationProvider` : `location.py`

Cours 5 : Programmation orientée objet

Le module `location` permet de manipuler des données de localisation et définit la classe abstraite `LocationProvider`. La classe `Location` décrit des objets contenant une latitude et une longitude. La classe `LocationSample` décrit des objets contenant un `datetime` (un objet `datetime` désignant une date et un temps), un objet `Location` et une `description` (par défaut une chaîne de caractères vide). D'autre part, elle redéfinit les opérateurs de comparaison pour refléter l'ordre chronologique (`ls1 <= ls2` si `ls1` est antérieur à `ls2`) de telle sorte qu'il est possible d'utiliser `sorted`, `min`, `max`, `<`, `>`, *etc.* Les diagrammes UML ci-dessous indiquent la structure générale de ces classes ainsi que les noms des méthodes à implémenter.

📖 Instructions :

1. 🛠️ **Urgent** : Définir la classe `Location` et son constructeur. Réutiliser le code implémenté pendant les séances de TPs (TP3).
2. Implémenter les *getters*.
3. Redéfinir la méthode `__str__` de sorte à afficher une localisation **exactement** sous la forme suivante et limiter le nombre de décimale à 5.
`Location [latitude: 48.85479, longitude: 2.34756]`
4. Définir (sans les implémenter pour l'instant, voir [Section X](#)) les méthodes `get_name` (qui renvoie la description textuelle du lieu correspondant à un objet `Location`) et `get_travel_distance_and_time` (qui renvoie le couple distance et temps de parcours de la localisation pour atteindre le lieu correspondant à un autre objet `Location`). Ignorer (pour l'instant) tous les attributs et méthodes contenant le mot "*api*".
5. 🛠️ **Urgent** : Définir la classe `LocationSample`.
6. Implémenter les *getters*.
7. Redéfinir la méthode `__str__` de sorte à afficher un objet `LocationSample` sous la forme suivante.
`LocationSample [datetime: 2026-04-11 14:25:00, location: Location [latitude: 48.85479, longitude: 2.34756]]`
8. 🛠️ **Urgent** : Redéfinir les opérateurs de comparaison.
9. 📌 **Optionnel** Lors de la création d'un objet `LocationSample` l'attribut `description` est une représentation HTML (date, position, image, *etc.*).



La classe (abstraite) `LocationProvider` décrit des objets permettant de produire une liste d'objets `LocationSample`. Elle spécifie l'existence d'une méthode `get_location_samples` qui renvoie une liste d'objets `LocationSample` triés par ordre chronologique (qui n'est **pas** implémentée, c'est aux classes filles de l'implémenter). Le diagramme UML ci-dessous décrit la classe. La classe **fournit** la fonction `show_location_samples` permettant d'afficher le résultat de `get_location_samples` sur une carte.

📖 Instructions :

1. Définir la classe abstraite `LocationProvider` en utilisant le module `abc`.

2. Spécifier l'existence de `get_location_samples` (méthode abstraite) en utilisant une annotation (un décorateur).
3. Implémenter la méthode `print_location_samples` en utilisant `get_location_samples`. Cette méthode doit afficher une chaîne de caractères décrivant, sous la forme suivante, les objets `LocationSamples` renvoyés par la méthode `get_location_samples`.

```
['LocationSample [datetime: 2026-03-11 12:25:00, location: Location [latitude: 48.85479, longitude: 2.34756]]', 'LocationSample [datetime: 2026-03-11 14:56:05, location: Location [latitude: 46.51774, longitude: 6.63223]]']
```
4.  **Urgent** : Implémenter la méthode `get_surrounding_temporal_location_samples` qui renvoie les objets `LocationSample` de la trace (obtenue via `get_location_samples`) juste avant et juste après le `datetime` passé en paramètre (la dernière information de localisation antérieure au `datetime` et la première postérieure au `datetime`).

Conseil

Créer la liste des objets `LocationSample` antérieurs au `datetime` passé en paramètre (utiliser `filter` ou une liste en compréhension) et en extraire le `LocationSample` le plus récent. De même, créer la liste des objets `LocationSample` postérieurs au `datetime` et en extraire le `LocationSample` le plus ancien.
 Une autre solution consiste à itérer sur la liste des objets `LocationSample` tant que les objets `LocationSample` rencontrés sont antérieurs au `datetime` passé en paramètre. Lorsqu'on rencontre le premier `LocationSample` postérieur au `datetime`, on peut identifier les `LocationSample` juste avant et juste après le `datetime`.

Attention

- Pour que cette dernière méthode fonctionne correctement, la liste renvoyée par la méthode `get_location_samples` doit être triée par ordre chronologique.
- Gérer le cas où il n'y a pas de `LocationSample` avant ou après (une des valeurs vaut `None` dans ce cas).

Exemple

Soit :

```
sample1 = LocationSample(datetime(2026,3,11,12,25), Location(46.517767,
↳ 6.63211))
sample2 = LocationSample(datetime(2026,3,11,14,56,5), Location(46.517738,
↳ 6.632233))
sample3 = LocationSample(datetime(2026,3,11,16,3), Location(46.517742,
↳ 6.63222))
```

La méthode `get_surrounding_temporal_location_sample(datetime(2026, 3, 11, 14, 42))` doit renvoyer `(sample1, sample2)`.

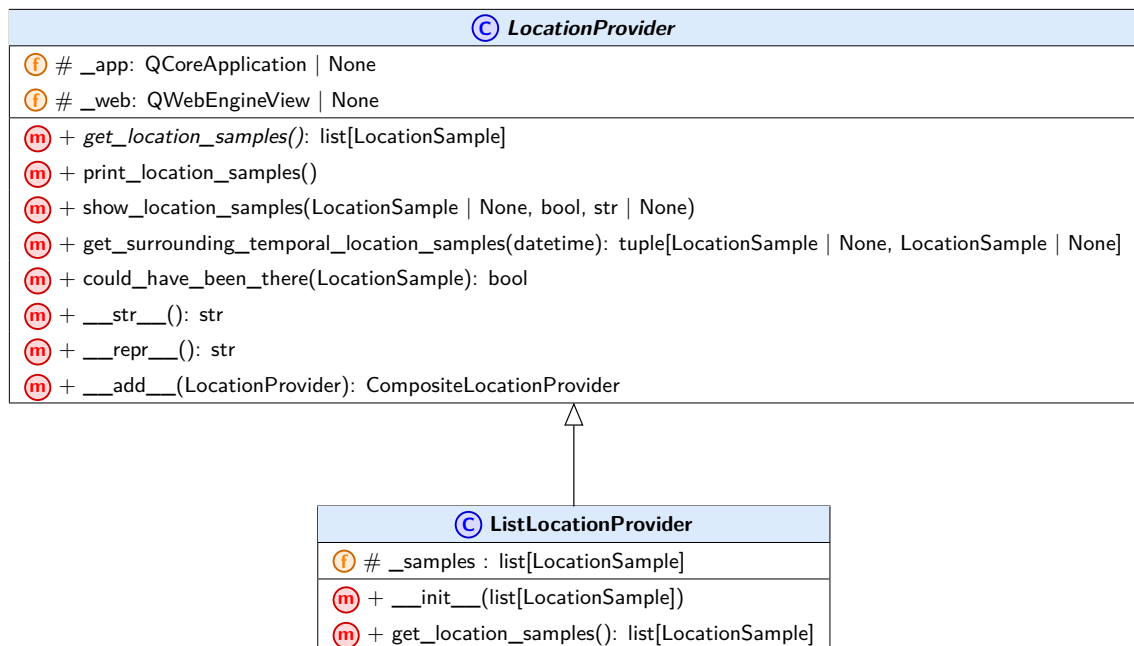
5. Définir la méthode `could_have_been_there` qui prend un `LocationSample` en paramètre et à l'aide de la méthode `get_surrounding_temporal_location_samples` détermine si un suspect a pu se rendre sur la scène du crime.

Conseil

Comparer le temps nécessaire pour rejoindre le `LocationSample` et le temps réellement écoulé.

6. Redéfinir la méthode `__str__` de sorte à afficher un objet `LocationProvider` sous la forme suivante.

```
'LocationProvider (5 location samples)'
```

7. **Urgent** : Implémenter la classe `ListLocationProvider`, héritant de `LocationProvider`, qui contient (dans un attribut) une liste d'objets `LocationSample` triés et dont la méthode `get_location_samples` renvoie cette liste (utiliser `deepcopy` pour éviter les modifications).
8. Vous pouvez utiliser la classe `ListLocationProvider` pour tester la classe abstraite `LocationProvider`. Tester l'affichage d'une liste de `LocationSample` avec la méthode `show_location_samples`.

Section V La classe `Suspect` : `suspect.py` Cours 9 : Encodage et formats de données

La classe `Suspect` décrit des objets qui contiennent les informations décrites dans le fichier *XML/JSON*, à savoir un nom de suspect-e et des sources de `LocationProvider`. On donne un exemple de fichier *XML* ci-dessous.

Instructions :

1. Définir la classe `Suspect` et son constructeur.
2. Implémenter les getters.
3. Optionnel Redéfinir la méthode `__str__` de sorte à afficher un objet `Suspect` sous la forme suivante.

```
[Suspect] Name: jdoe, Location provider: PictureLocationProvider (source:
'../data/pics/jdoe' (JPG, JPEG, jpg, jpeg), 2 location samples)
```
4. Implémenter une méthode `create_suspects_from_xml_file` qui prend en paramètre un nom de fichier et qui parse le fichier *XML* et crée la liste de suspect-e-s correspondante. De quel type de méthode s'agit-il (instance / classe / statique) ?
 - (a) Extraire l'arbre *XML* du fichier et le parcourir à l'aide du module `xml.etree.ElementTree`.
 - (b) Pour chaque élément suspect dans l'arbre *XML*, créer un objet `Suspect`.
 - (c) Pour chaque élément source dans l'arbre *XML*, créer l'objet `LocationProvider` correspondant (en fonction du type de `LocationProvider` ; élément "type"). Tous les paramètres nécessaires peuvent être trouvés dans les sous-éléments de l'élément source.
 - (d) Regrouper en un seul objet `LocationProvider`, **tous** les objets `LocationProvider` créés à partir des différentes sources ; utiliser des objets `CompositeLocationProvider` en utilisant le constructeur de la classe ou directement avec la notation "+" (voir [Section VI](#)).

⚠ Attention

Les noms de fichiers contenus dans le fichier XML sont exprimés relativement au fichier XML lui-même. C'est-à-dire que si le fichier "data/suspects.xml" contient un élément dir : "../data/pics/jdoe", le chemin réel du répertoire est "data/../data/pics/jdoe" (= "data/pics/jdoe"). Gérer cette situation en utilisant les fonctions `joinpath` et `resolve` du module `Path`.

> Exemple

- `Path(filename).parent` renvoie le chemin du répertoire correspondant au fichier *filename*. `Path("data/suspects.xml").parent` retourne "data".
- `dirname.joinpath(path)` renvoie la concaténation des deux chemins. `Path("data").joinpath("../data/pics/jdoe")` retourne `data/../data/pics/jdoe`.
- `path.resolve()` renvoie le chemin *path* sans les séparateurs inutiles ("data/../data/" devient "data"). `Path("data/../data/pics/jdoe").resolve()` retourne "data/pics/jdoe".

Alternative : implémenter une méthode similaire `create_suspects_from_json_file` opérant sur un fichier *JSON*.

- (e) Gérer les exceptions : si une exception survient lors de la création d'un suspect-e (ou d'une source qui lui est associée), afficher l'erreur et continuer l'exécution pour les autres suspect-e-s/sources. Pour afficher une erreur, afficher simplement un message (sur la sortie standard d'erreur `sys.stderr`) de la forme (uniquement si l'utilisateur-trice a spécifié l'argument `--verbose` dans la ligne de commande ; utiliser l'objet configuration pour ceci, voir [Section III](#)) :

```
Attention: Une erreur est survenue pendant le traitement de la source ([Errno
↪ 2] Aucun fichier ou répertoire : '../data/../data/pics/jdoe'):
<source>
<type>Photographs</type>
<dir>../data/pics/jdoe</dir>
</source>
```

© Suspect	
📄	- __name : str
📄	- __lp : LocationProvider
🔧	+ __init__(str, LocationProvider)
🔧	+ get_name(): str
🔧	+ get_location_provider(): LocationProvider
🔧	+ __str__(): str
🔧	+ __repr__(): str
🔧	+ create_suspects_from_xml_file(str): list[Suspect]
🔧	+ create_suspects_from_json_file(str): list[Suspect]

> Exemple

Exemple de fichier XML :

```
<?xml version="1.0"?>
<suspects>
  <suspect>
    <name>Jane Doe</name>
    <sources>
      <source>
        <type>Photographs</type>
        <dir>../data/pics/jdoe</dir>
      </source>
    </sources>
  </suspect>
</suspects>
```

```

    <source>
      <type>Wi-Fi</type>
      <db>../data/db/wifi.db</db>
      <username>jdoe</username>
    </source>
  </sources>
</suspect>
</suspects>

```

Note : pour tester plus rapidement les autres parties du code, créer un suspect manuellement plutôt que d'implémenter la méthode de création à partir d'un fichier *XML/JSON*, par exemple :

```

lp1 = PictureLocationProvider('../data/pics/jdoe')
lp2 = WifiLogsLocationProvider('../data/db/wifi.db', 'jdoe')
lp = CompositeLocationProvider(lp1, lp2)
miranda = Suspect("Jane Doe", lp)

jdoe.get_name() doit retourner 'Jane Doe'.
jdoe.get_location_provider() doit retourner :

CompositeLocationProvider (10 location samples)
+   PictureLocationProvider (source: '../data/pics/jdoe' (JPG, JPEG, jpg, jpeg), 6
→   location samples)
+   WifiLogsLocationProvider (source: '../data/db/wifi.db', utilisateur 'jdoe', 4
→   location samples)

```

Section VI La classe CompositeLocationProvider : location.py

Cours 6 : UML et Patrons de conception

Compléter le module `location` en ajoutant la classe `CompositeLocationProvider`. Le modèle de `LocationProvider` composite permet de manipuler plusieurs objets `LocationProvider` comme s'il s'agissait d'un seul. La classe `CompositeLocationProvider` implémente ce modèle, selon le patron de conception composite, pour deux objets `LocationProvider`. Le diagramme *UML* ci-dessous décrit la classe.

Instructions :

1. Définir la classe `CompositeLocationProvider` et son constructeur.
2. Implémenter la méthode `get_location_samples`.

Conseil

Penser à garder l'ordre chronologique dans la liste de `LocationSample` renvoyée.

3. **Optionnel** Redéfinir la méthode `__str__` de sorte à afficher un objet `CompositeLocationProvider` sous la forme suivante : l'affichage des deux objets `LocationProvider` contenu dans un composite est décalé vers la droite. On utilisera la fonction `indent` définie dans le module `util` (voir [Section I](#)).

```

CompositeLocationProvider (6 location samples)
+ ListLocationProvider (2 location samples)
+ CompositeLocationProvider (4 location samples)
  + ListLocationProvider (2 location samples)
  + ListLocationProvider (2 location samples)

```

4. Définir une méthode appropriée qui renvoie un nouveau composite lorsqu'on utilise l'opérateur `+` pour additionner deux instances de `LocationProvider`.

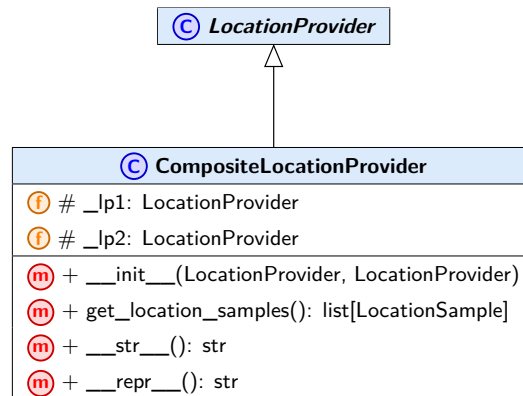
Par exemple : `lp1 + lp2` renvoie un composite dont les composants sont `lp1` et `lp2`.

Attention

Placer judicieusement cette méthode dans la bonne classe du module `location`.

>_ Exemple

L'expression `lp1 + lp2` renvoie un objet `CompositeLocationProvider` composé de `lp1` et de `lp2`.



Section VII Le module de sources photographiques : `picture.py` Cours 3-5 : Programmation orientée objet

Pour certain-e-s suspect-e-s, les enquêteur-trice-s ont retrouvé sur leurs disques durs certaines de leurs photos (prises avec un smartphone) qui contiennent les dates et la localisation auxquelles les photos ont été prises (données *EXIF*³). On définit une classe `PictureLocationProvider` pour manipuler de telles sources de données. Les objets de cette classe sont construits à partir de fichiers image au format JPEG - avec pour extension `.jpg`, `.jpeg`, `.JPG` ou `.JPEG` — cette liste est stockée comme un attribut de classe) contenu dans le répertoire *directory* passé en paramètre du constructeur. La date/heure et la localisation des photos sont extraites des informations *EXIF* (grâce au module `ExifRead`); la majeure partie du code permettant d'extraire les données d'une image est fournie : il ne reste que quelques lignes à compléter. Le diagramme UML ci-dessous décrit la classe. La méthode statique `_convert_to_degrees` est utilisée pour l'extraction des données EXIF (elle est fournie).

Instructions :

1. Définir la classe `PictureLocationProvider` et les éventuels attributs de classe (ainsi que les éventuels getters pour y accéder).

Attention

Faire en sorte que la liste des extensions ne puisse pas être altérée via le résultat de la méthode `get_valid_extensions`.

2. Compléter la méthode `_extract_location_sample_from_picture` pour renvoyer le triplet : `timestamp (datetime)`, `latitude (float)`, `longitude (float)` extrait de l'image. Le code pour les coordonnées est déjà fourni. Il ne reste qu'à transformer la date/heure de la prise de vue en un `datetime`. Utiliser la méthode `strptime` du module `datetime`.

Attention

La date d'une photo est exprimée selon la timezone UTC.

3. Pour plus d'informations veuillez consulter : [ce lien](#)

Conseil

- Afficher la date extraite des données *EXIF* pour en connaître le format.
- Créer un objet `datetime` à partir de la date extraite (si elle est non `None`) en utilisant la méthode `strptime`. Vous pouvez utiliser le *debugger* afin de regarder le contenu des différentes variables.

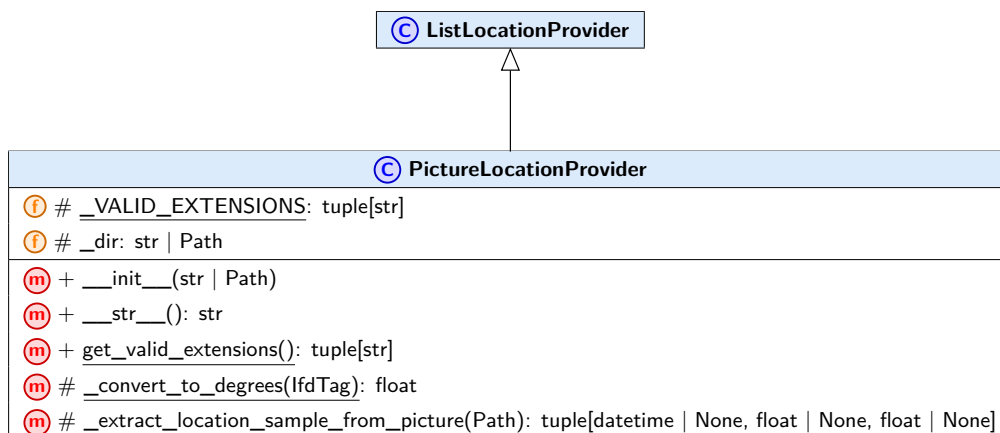
3. Définir son constructeur : dans le constructeur, on construit une liste d'objets `LocationSample`, en extrayant, à l'aide de la méthode `_extract_location_sample_from_picture` précédemment complétée, les informations de chaque image du répertoire passé en paramètre (`directory`). Plus précisément, cette méthode renvoie un `datetime`, une latitude et une longitude, à partir desquels il faut créer un objet `LocationSample`. Si un des éléments renvoyés par la méthode `_extract_location_sample_from_picture` est `None`, ne pas ajouter l'élément à la liste de `sample`. Utiliser la méthode `iterdir` de la classe `Path` (module `pathlib`) pour parcourir les fichiers et dossiers du répertoire (voir la slide n°32 du cours n°2). La liste de `sample` est ensuite stockée grâce à l'attribut défini dans la classe `ListLocationProvider`.

4. Modifier le constructeur pour intercepter les exceptions et continuer l'exécution en passant au fichier suivant en cas de problème. Afficher simplement un message de la forme suivante (uniquement si l'utilisateur·trice a spécifié l'argument `--verbose`):

Attention: fichier ignoré 'unil.jpg' (Informations de temps et/ou location manquantes)

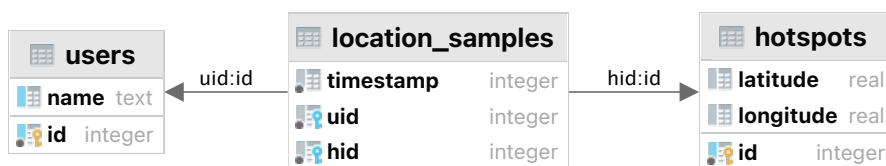
5. **Optionnel** Redéfinir la méthode `__str__` de sorte à afficher un objet `PictureLocationProvider` sous la forme suivante :

`PictureLocationProvider (source: '../data/pics/jdoe' (JPG, JPEG, jpg, jpeg), 2 location samples)`



Section VIII 📶 Le module de sources Wi-Fi : `wifi.py` Cours 10 : Bases de données

Les enquêteur·trice·s ont obtenu accès aux logs de connexion du réseau Wi-Fi de l'UNIL (via RADIUS/EAP). À chaque fois qu'un smartphone se connecte au réseau eduroam (il se connecte automatiquement dès lors qu'un point d'accès est à portée s'il est configuré pour utiliser ce réseau), un événement est créé dans les logs. L'événement contient l'identifiant UNIL de l'utilisateur·trice et l'identifiant du point d'accès Wi-Fi. La localisation de ces points d'accès est connue. Toutes ces données sont stockées dans une base de données au format *SQLite* suivant ce schéma.



La base de données contient trois tables décrites dans le diagramme UML ci-dessous. La table `users` contient la liste des utilisateur·trice·s spécifiés par un identifiant unique et un nom d'utilisateur·trice (unique

également, par exemple `jdoe`). La table `hotspots` contient la liste des points d'accès Wi-Fi spécifiés par un identifiant unique, une latitude et une longitude. La table `location_samples` contient les événements de connexion spécifiés par un `timestamp` et une référence (via les clés étrangères correspondantes) vers l'utilisateur·trice et le point d'accès concerné. On définit une classe `WifiLogsLocationProvider` pour manipuler de tels logs comme une source d'objets `LocationSample`, c'est-à-dire un location provider. Un tel objet est construit à partir du nom du fichier de base de données et le nom d'utilisateur·trice UNIL de la personne suspect·e.

Instructions :

1. Définir la classe `WifiLogsLocationProvider`.
2. Définir son constructeur. Dans le constructeur, construire une liste d'objets `LocationSample` à partir des données contenues dans la base de données passée en paramètre. Pour ce faire, utiliser une commande *SQL* adéquate (utiliser une requête de type **SELECT ... JOIN ON ...** pour combiner les données des différentes tables). La liste de `sample` est ensuite stockée grâce à l'attribut défini dans la classe `ListLocationProvider`. On peut utiliser la fonction `dict_factory` (donnée en cours et pendant les séances de TPs - Semaine 11, Base de données) comme `row_factory` pour la connexion afin de pouvoir accéder directement aux résultats de la requête à l'aide des noms de colonnes de la base de données *SQLite* (c.f. TP).

⚠ Attention

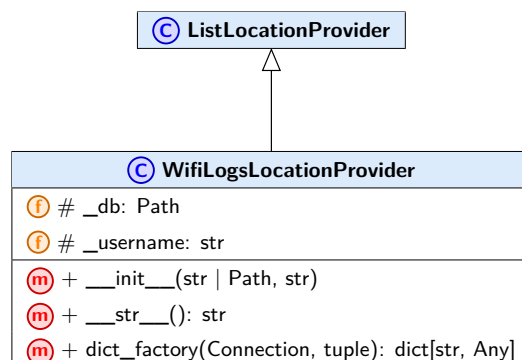
La date d'une source Wi-Fi est exprimée selon la timezone UTC+2, l'heure locale à l'UNIL.

💡 Conseil

- Ecrire une requête *SQL* pour récupérer les données voulues en effectuant deux opérations **JOIN** et un filtre **WHERE** (pour sélectionner l'utilisateur·trice dont on veut récupérer les logs ; le nom de l'utilisateur·trice est passé en paramètre au constructeur).
- Créer une connexion à la base de données stockée dans le fichier dont le nom est passé en paramètre au constructeur.
- Récupérer un curseur et exécuter la requête.
- Récupérer les résultats, créer un objet `LocationSample` pour chaque enregistrement et l'ajouter à la liste de `sample`.

3. Modifier la requête *SQL* utilisée pour trier directement les résultats par ordre chronologique.
4. **Optionnel** Redéfinir la méthode `__str__` de sorte à afficher un objet `WifiLogsLocationProvider` sous la forme suivante :

```
WifiLogsLocationProvider (source: '../data/db/wifi.db', user 'jane', 3
location samples)
```



Les enquêteur·trice·s ont obtenu accès aux logs des téléphones portables de certain·e·s suspect·e·s. À chaque fois qu'une action est effectuée avec le smartphone, l'événement est inséré dans les logs du téléphone. De nombreux événements y figurent mais ceux qui sont intéressants pour l'enquête sont ceux qui sont inscrit lorsque certaines applications demandent des données de géolocalisation. Ainsi, certaines lignes de logs contiennent l'heure de la demande ainsi que les coordonnées courantes de l'utilisateur·trice.

Chaque fichier contient des logs (un par ligne). Chaque ligne débute par l'heure exacte où le log a été créé. Le texte y figurant ensuite dépend de l'action effectuée. Les logs de requête de coordonnées ont plusieurs particularités :

- Ils contiennent le nom de l'application effectuant la requête
- Ils contiennent des coordonnées
- Ils contiennent la source de ces coordonnées (GPS, WIFI, ...)

Aucune de ces particularités n'est exclusive mais si un log les contient toutes, il s'agit bien d'un log de requête de géolocalisation. Attention toutefois à plusieurs éléments. Certains logs n'ont pas pu obtenir le timestamp, il est donc fixé à UNKNOWN. D'autres n'ont pas pu obtenir de coordonnées, elles sont donc également fixées à UNKNOWN. Quant aux sources de coordonnées, certaines sont trop peu précises, par conséquent, seule les données provenant d'une **source GPS** seront considérées (3ème ligne de l'exemple ci-dessous).

> Exemple

```
[2026-03-17T15:50:00.000] Location request from snapchat returned coordinates:
→ (46.51338672040743, 6.590572670701977), source: cellular
[2026-03-17T15:50:30.000] Location request from instagram returned coordinates:
→ ('46.51563775632543', '6.572046754615044'), source: UNKNOWN
[2026-03-17T15:51:00.000] Location request from netflix, returned coordinates:
→ (46.5228407, 6.5811926), source: GPS
[2026-03-17T15:51:30.000] Location request from whatsapp returned coordinates:
→ ('46.531384909328004', '6.572975069761746'), source: wi-fi
```

☰ Instructions :

1. Définir la classe `LogsLocationProvider`.
2. Définir son constructeur. Dans le constructeur, définir un attribut contenant le nom du fichier de logs. Construire une expression régulière répondant aux critères ci-dessus. Utiliser ensuite cette expression pour filtrer les logs qui font partie du langage qu'elle définit.
3. Extraire de ces logs valides les données temporelles et de géolocalisation. Créer ensuite un `LocationSample` et l'ajouter dans la liste. Pour ce faire, il faut extraire un datetime, une latitude et une longitude d'un log donné.

⚠ Attention

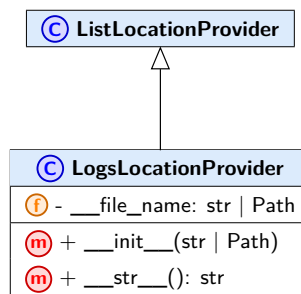
La date des logs est exprimée selon la timezone UTC+2.

💡 Conseil

Chaque ligne de fichier `log` se termine par un retour chariot (`\n`). En tenir compte lors de l'analyse des logs ou exclure ce retour à la ligne en utilisant la méthode `strip()`.

4. **Optionnel** Redéfinir la méthode `__str__` de sorte à afficher un objet `LogsLocationProvider` sous la forme suivante :

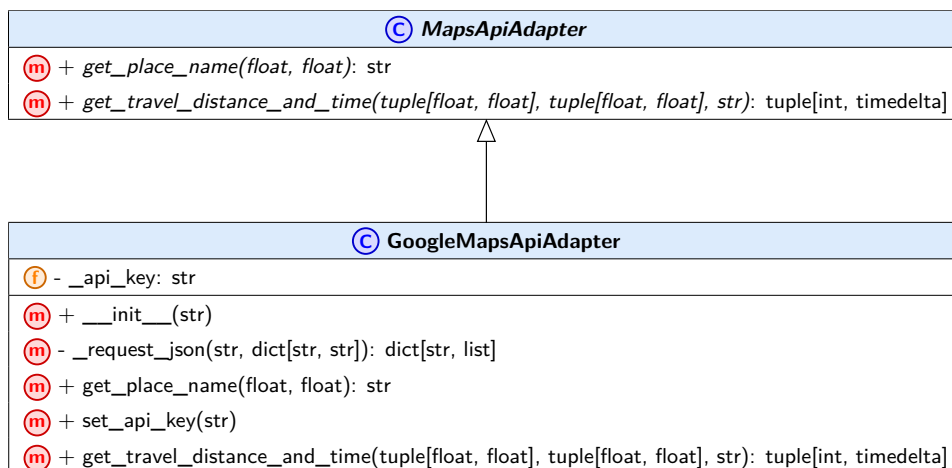
```
LogsLocationProvider (source: ../data/logs/jdoe.log, 2 location samples)
```



Section X Le module de cartographie Google Maps : location.py

Cours 11 : Programmation réseau et APIs web

Enrichir la classe `Location` du module `location` (voir [Section IV](#)) grâce au service de cartographie Google Maps (via son API web), auquel on accède en construisant des requêtes HTTP avec le module `urllib`. Ce module ajoute la possibilité d'interroger l'API web de Google Maps afin d'obtenir l'adresse textuelle ou formatée correspondante à des coordonnées géographiques (reverse geocoding, comme vu en TP), ainsi que d'obtenir la distance et le temps nécessaire pour effectuer un trajet (à pied ou en utilisant un autre mode de transport) entre deux points.



Instructions :

1. Définir une classe abstraite `MapsApiAdapter` qui décrit l'interface commune d'un SIG, avec les méthodes `get_place_name(latitude, longitude)`, qui renvoie une chaîne de caractères décrivant le lieu correspondant aux coordonnées fournies en paramètre, et `get_travel_distance_and_time(origin, destination, mode="walking")`, qui renvoie un tuple contenant la distance et la durée du trajet entre les coordonnées fournies en paramètre supposant que l'on se déplace à pied.
2. Implémenter une classe `GoogleMapsApiAdapter`, héritant de `MapsApiAdapter`, chargée d'interroger l'API web de Google Maps. Cette classe doit stocker la clé d'API et fournir une méthode utilitaire privée, appelé `_request_json`, qui construit l'URL, effectue la requête HTTP avec `urllib` et convertit la réponse JSON. Cette classe possède un setter permettant de modifier l'attribut stockant la clé API. Pour obtenir une clé d'API Google, on publiera une annonce sur Moodle contenant des instructions à cette fin.

Attention

Pour construire l'URL, encoder les paramètres de l'URL en utilisant la fonction `urlencode` (c.f. diapositive 20 du cours Réseaux et APIs Web). N'oubliez pas d'inclure une paire clé-valeur correspondant à la clé API (c.f. [exemple de paramètres distance](#) [matrice](#) ou [geocoding](#)). Inclure toujours `context=SSL_CONTEXT` comme paramètre à la

fonction `urlopen`. Ne pas utiliser le module `googlemaps`.

3. Dans `GoogleMapsApiAdapter`, implémenter `get_place_name` en utilisant `urllib` pour interroger l'API Google reverse geocoding,⁴ puis en extrayant le champ `formatted_address` de la réponse JSON. Renvoyer une valeur par défaut et afficher un message d'erreur en cas de problème.
Optionnel Spécifier la langue dans laquelle exprimer le nom du lieu.
4. Dans `GoogleMapsApiAdapter`, implémenter `get_travel_distance_and_time` en utilisant `urllib` pour interroger l'API Google distance matrix⁵, puis en extrayant le couple distance/temps de parcours depuis la réponse JSON. Utiliser le paramètre adéquat pour obtenir les résultats dans le système international (pour les unités de mesure de la distance par exemple).
Optionnel Ajouter un paramètre optionnel pour spécifier le mode de transport.
5. Adapter ensuite la classe `Location` pour qu'elle s'appuie sur cette abstraction plutôt que d'appeler directement l'API. Ajouter des attributs de classe pour stocker l'adaptateur courant, ainsi qu'une méthode de classe `set_maps_adapter` qui affecte un objet `MapsApiAdapter` fourni comme paramètre à l'attribut correspondant de la classe `Location`.
6. Ajouter une méthode privée (pour usage interne) `_check_api_init` qui permet de vérifier que l'adaptateur cartographique a bien été initialisé, avec une explication de la marche à suivre si ce n'est pas le cas.

>_ Exemple

Exception: Spécifier une Adapter valide pour utiliser l'API du SIG. Utiliser `Location.set_maps_adapter(...)`

7. Implémenter les méthodes d'instance `get_name` et `get_travel_distance_and_time` de `Location` de sorte qu'elles délèguent le travail à l'objet `MapsApiAdapter` configuré. Avant chaque requête dans la classe `GoogleMapsApiAdapter` vérifier que l'API est correctement initialisée.



Conclusion

Grâce à ces modules, vous avez maintenant la possibilité de collecter et d'analyser les traces de mobilité des différents suspect-e-s. Récupérer la liste des samples pour écarter les suspect-e-s innocents, n'ayant pas pu avoir le temps de se rendre sur le lieu du crime. À vous de jouer !

4. [Informations détaillées sur l'API](#)

5. [Informations détaillées sur l'API](#)